



Lingüística Matemática

1

Strings y Lenguajes

Vamos a introducir el sistema matemático que sirve como nuestro modelo abstracto de lenguaje. Como abstracción de lenguaje vamos a emplear conjuntos que tienen "strings" como miembros. A continuación vamos a desarrollar las propiedades matemáticas básicas de estas abstracciones.

Strings y Operaciones con Strings

Alfabeto: Conjunto finito de símbolos que representa los objetos o elementos atómicos (o sea no divisibles) que se emplean en la construcción de sentencias. Lo denominaremos con la letra Σ .

String de longitud k sobre un alfabeto Σ : Es una secuencia de k símbolos en Σ .

$$\omega = v_1, v_2, v_3, \dots, v_k \text{ tal que } v_i \in \Sigma$$

Comúnmente vamos a omitir las comas y a escribir ω como

$$\omega = v_1 v_2 v_3 \dots v_k$$

Vamos a reservar letras griegas minúsculas para indicar strings.

Indicamos que un string ω tiene longitud k mediante:

$$|\omega| = k$$

String Vacío: Es el único string que no contiene símbolos y es indicado con la letra λ . Luego

$$|\lambda| = 0$$

Ejemplo: Sea $\Sigma = \{a, b, c\}$, luego $\omega = abcbcb$ es un string en Σ y $|\omega| = 5$.

Conjunto W : definiremos W_k como el conjunto de todos los strings de longitud k sobre un alfabeto Σ :

$$W_k = \{\omega / |\omega| = k\}$$

W_0 es el conjunto que contiene el string vacío, empleamos λ para indicar este conjunto.

$$W_0 = \lambda = \{\lambda\}$$

El conjunto

$$W = \bigcup_{k=0}^{\infty} W_k$$



es el conjunto de todos los strings definidos sobre Σ , incluyendo el string vacío.

Concatenación: Operación también denominada multiplicación de strings, es una operación binaria definida sobre el conjunto W :

$$m: W \times W \rightarrow W$$

Dados dos strings ω y φ tales que:

$$\omega = v_1 v_2 v_3 \dots v_m; \quad \varphi = u_1 u_2 u_3 \dots u_n$$

La concatenación de los strings ω y φ es la secuencia de símbolos formada al extender la secuencia ω con la secuencia φ .

$$m(\omega, \varphi) = v_1 v_2 v_3 \dots v_m u_1 u_2 u_3 \dots u_n$$

A la operación $m(\omega, \varphi)$ la vamos a indicar también $\omega \cdot \varphi$ o simplemente $\omega\varphi$.

Si definimos un string ω como $\omega = \varphi\psi$ decimos que φ es el prefijo de ω , y si ψ no es el string vacío decimos que ψ es el prefijo propio de ω .

Propiedades de la operación concatenación

1. $\forall \omega, \varphi \in W; \omega \cdot \varphi \in W$. Luego el conjunto W es cerrado bajo la operación concatenación.
2. $\forall \omega, \varphi, \psi \in W; \omega \cdot (\varphi \cdot \psi) = (\omega \cdot \varphi) \cdot \psi$. Cumple la propiedad Asociativa.
3. Para cada $\omega \in W$, $\omega \cdot \lambda = \lambda \cdot \omega = \omega$. Luego el string vacío se comporta como un elemento identidad.
4. No es necesariamente cierto que $\omega \cdot \varphi = \varphi \cdot \omega$. La concatenación no es conmutativa.
5. $\forall \omega, \varphi \in W; |\omega \cdot \varphi| = |\omega| + |\varphi|$.

ω^k : La notación ω^k es empleada para indicar la concatenación de k copias del string ω .

$$\omega^k = \omega \cdot \omega \cdot \omega \dots \omega$$

$k \text{ veces}$

El **reverso** del string $\omega = v_1 v_2 v_3 \dots v_n$ es el string $v_n \dots v_3 v_2 v_1$ que se indica ω^R .

Cada string que es definido sobre un alfabeto Σ es una *secuencia finita* de símbolos, no existe un objeto que sea un string infinito. Si bien a veces vamos a emplear el término secuencia para referirnos a un "string", nunca vamos a emplear el término "string" para referirnos a una secuencia infinita.

En consecuencia, el conjunto de todos los strings definidos sobre un alfabeto Σ es enumerable.

Propuesta: Si Σ es un alfabeto no vacío, luego el conjunto W de todos los strings definidos sobre Σ es *enumerable*.

Un conjunto cualquiera X se dice *enumerable* si sus elementos pueden ser ubicados en una relación de correspondencia uno a uno con los enteros positivos.

I.2. Operaciones con Lenguajes

Lenguaje definido sobre un alfabeto Σ es cualquier conjunto de strings definidos sobre el conjunto Σ . Así, cada lenguaje definido en Σ es un subconjunto del conjunto W de todos los strings en Σ . Cada lenguaje es un conjunto o set contable.

Recordatorio: Se dice que un conjunto es contable si es finito o enumerable.

La clase de todos los lenguajes definidos en el alfabeto Σ es el conjunto o clase de todos los subconjuntos de W , es decir, el conjunto potencia de W , que se simboliza:



$\wp(W)$ conjunto potencia de W

$\wp(W)$ es incontable.

Ejemplo: Sea $\Sigma = \{a\}$ un alfabeto con un solo símbolo. Luego en el lenguaje

$$L = \{a^k / k \geq 0\}$$

cada string está compuesto por a 's:

$$L = \{\lambda, a, aa, aaa, aaaa, \dots\}$$

Luego, el conjunto L es enumerablemente infinito, aun cuando cada uno de sus elementos es una secuencia finita.

Las operaciones básicas que se aplican a lenguajes son también las operaciones básicas aplicables a conjuntos, es decir unión, intersección y complemento relativo a W .

Concatenación de Lenguajes

Dados dos lenguajes A y B , definimos a la concatenación de los mismos

$$A.B = \{\omega.\varphi / \omega \in A, \varphi \in B\}$$

al conjunto de strings que resulta de extender cada string definido en A con cada string definido en B .

$A.B$ también puede ser escrito como AB .

Ejemplo: Dados $A = \{a, ab\}$, $B = \{c, bc\}$, luego $AB = \{ac, abc, abbc\}$

Propiedades de la concatenación

1. El conjunto $\lambda = \{\lambda\}$ es un elemento identidad en la concatenación de lenguajes.
Si $A = \{a, bc\}$, $\lambda = \{\lambda\}$, luego $A.\lambda = \{a.\lambda, bc.\lambda\} = \{a, bc\} = A$
2. El conjunto $\emptyset$ es el elemento absorbente.
 $A.\emptyset = \{\omega.\varphi / \omega \in A, \varphi \in \emptyset\} = \emptyset$
dado que $\emptyset$ no posee elementos. De forma similar se puede probar que $\emptyset.A = \emptyset$. Luego debemos distinguir entre el conjunto vacío y el conjunto que contiene al string vacío.
3. Propiedad Asociativa: $A.(B.C) = (A.B).C$
4. Concatenación de lenguajes no necesariamente es conmutativa:
 $A.B$ necesariamente igual a $B.A$
5. La concatenación distribuye sobre la unión: $A.(B \cup C) = A.B \cup A.C$
6. La concatenación **no** distribuye sobre la intersección: $A.(B \cap C) \neq A.B \cap A.C$

Clausura de un Lenguaje

Sea Σ un alfabeto y W el universo de strings generados a partir de Σ .

Para un lenguaje $A \subseteq W$, definimos los conjuntos A^k , $k \geq 0$ mediante:

$$\begin{aligned} A^0 &= \{\lambda\} \\ A^{k+1} &= A^k.A \end{aligned}$$



Por ejemplo:

$$\begin{aligned} A^2 &= A.A \\ A^3 &= A^2.A \\ A^4 &= A^3.A \\ &\vdots \\ A^{k+1} &= A^k.A \end{aligned}$$

Luego cada uno de estos conjuntos A^k es un subconjunto de W dado que W es cerrado a la operación concatenación.

Luego la unión de todos estos conjuntos:

$$A^* = \bigcup_{k=0}^{\infty} A^k$$

es también un subconjunto de W y es llamado clausura de A . El operador $*$ se conoce como estrella de Kleene (Kleene Star). La clausura de un lenguaje contiene cada string que pueda ser formado mediante la concatenación de un número arbitrario de strings tomados del lenguaje.

Ejemplos:

1. Sea $A = \{a, ab\}$ sobre $\Sigma = \{a, b\}$

Luego

$$A^3 = A.A.A = \{aaa, aaab, aaba, aabab, abaa, abaab, ababa, ababab\}$$

Luego podemos ver que

$$\{a, ab\}^3 \neq \{a^3, (ab)^3\}$$

2. Sea $A = \{a\}$, $B = \{0, 1\}$, luego

$$A^* = \{\lambda, a, aa, aaa, aaaa, \dots\} = \{a^k / k \geq 0\}$$

$$B^* = \{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, \dots\}$$

A^* es el conjunto de todos los strings posibles definidos sobre A , lo mismo es válido para B .

Definición: Dado que el conjunto de todos los strings de longitud $k + 1$ definidos sobre un alfabeto Σ puede ser escrito como:

$$W_{k+1} = W_k.V$$

se deduce que:

$$W_k = V^k$$

y que:

$$W = V^*$$

Luego Σ^* es el universo de strings que pueden ser definidos a partir del alfabeto Σ .

Reverso de un Lenguaje L

El reverso de un lenguaje L contiene el reverso de cada string definido en L .

$$L^R = \{\omega^R / \omega \in L\}$$

Si cada string de L es a su vez su propio reverso, luego tendremos que:

$$L = L^R$$

Sin embargo, la afirmación inversa no es necesariamente cierta, es decir si $L = L^R$, ello no implica que cada uno de los strings del mismo sea igual a su reverso.

Por ejemplo: $L = \{01, 10\}$; $L^R = \{10, 01\}$; si bien $L = L^R$, los strings $01 \neq 10$ y $10 \neq 01$.



Gramáticas Formales

Introducción

La traducción de texto de un lenguaje a otro lenguaje ha sido por largo tiempo el sueño de lingüistas y computadores científicos. Si bien luego de la introducción de lenguajes de alto nivel hubo un período de gran actividad en la búsqueda de formalizaciones precisas de lenguajes para apoyar y simplificar la descripción del proceso de traducción, los mayores frutos no se dieron en el área de traducción de lenguajes naturales. Sin embargo la teoría desarrollada tuvo y tiene una gran influencia en el diseño y procesamiento de lenguajes de programación.

A continuación presentamos los conceptos fundamentales de las gramáticas formales y su relación con los lenguajes que ellas describen. A pesar de que se han estudiado muchas clases de gramáticas y lenguajes, nuestra atención se va a concentrar en aquellas incluidas en la jerarquía propuesta por Chomsky.

Representaciones de Lenguajes

De acuerdo al diccionario, "un lenguaje es el agregado de palabras y métodos de combinar las mismas que es empleado por una determinada nación, comunidad, grupo étnico, religioso, etc.". Esta definición no provee una descripción matemática de un lenguaje. Los lingüistas piensan que cualquier formalismo que represente adecuadamente un lenguaje natural deberá poder representar la infinidad de sentencias que puedan aparecer en el uso del lenguaje. Sin embargo, no podemos esperar que todas las posibles sentencias del lenguaje sean enumeradas.

De la misma forma, y pasando a los lenguajes de programación, no podemos esperar que se listen todos los posibles programas a ser escritos por los usuarios de lenguajes como C, Python, Pascal, etc. Luego, el problema central en la descripción de un lenguaje es proveer una especificación finita para una clase de objetos esencialmente infinita.

Dado un alfabeto Σ , el conjunto Σ^* es la totalidad de todos los posibles strings definidos sobre Σ . Un lenguaje L definido sobre Σ es un subconjunto arbitrario de Σ^* . Ya dijimos que una descripción de L es suficiente si puede ser empleada para decidir cuándo un miembro dado de Σ^* es o no miembro de L .

En castellano, el alfabeto posee 29 letras que se pueden escribir en mayúsculas y minúsculas, tenemos 10 dígitos, el espacio, signos de puntuación, admiración, etc. Si existiese una gramática formal de este lenguaje, la misma debería poder discernir si una oración dada es correcta o no.

Para un lenguaje de programación, el alfabeto es la colección de símbolos no divisibles que pueden aparecer en el programa. Una sentencia es generalmente considerada un string formado por esos símbolos que representa un programa completo. Una gramática satisfactoria para un lenguaje de programación debe permitirnos discernir por un procedimiento mecánico si una secuencia arbitraria de símbolos es un programa "bien conformado".

El requerimiento de que la representación de un lenguaje sea finita parece ser tremendamente severa e implica casi directamente que muchos lenguajes no poseen una representación finita. El problema es si los lenguajes naturales pertenecen o no a esta categoría, sin embargo a nosotros por ahora no nos ocupa.

Una forma de representar un lenguaje ya fue sugerida y es la de emplear un *aceptor* del lenguaje. Si el aceptor posee un número finito de estados funcionalmente relevantes, él en sí mismo constituye una representación finita. Sin embargo, vamos a encontrar que las clases de lenguajes para los cuales estas máquinas de estado finito proveen una descripción estructural es bastante limitada, vamos a considerar otras máquinas abstractas más poderosas como posibles representaciones finitas de lenguajes.

La definición de lenguaje por medio de una máquina aceptora es un enfoque analítico a la representación del lenguaje, ya que la máquina verifica directamente si la sentencia de entrada es o no miembro del lenguaje.

Otra alternativa es emplear una representación generativa del lenguaje, es decir emplear un conjunto finito de reglas que si son seguidas en un orden válido van a permitir solamente la construcción de strings que sean sentencias válidas del lenguaje.

¿Puede una representación generativa de un lenguaje ser empleada para decidir si un string dado es miembro del lenguaje? Veremos que sí, este procedimiento funciona para todas las gramáticas, con excepción de la forma más general. Para esa forma general veremos que no existe procedimiento mecánico que pueda decidir si un dado string es generado por la gramática.

Conceptos básicos de Gramáticas



Un ejemplo del Castellano

Concepto clave * construcción de oraciones mediante la sucesiva aplicación de reglas gramaticales
Una regla específica cómo una frase puede ser reescrita concatenando sus frases constitutivas.

Jorge y Pablo subieron a la terraza

sujeto

predicado

La construcción de esta oración puede ser descripta mediante la regla gramatical

1.- <oración> → <sujeto><predicado>

La inclusión entre ángulos tiene el propósito de evitar cualquier posible confusión con palabras o símbolos que aparezcan en las oraciones del lenguaje que está siendo analizado.

En este ejemplo el sujeto es un par de sustantivos vinculados por una conjunción, o sea

<sujeto> → <sustantivo><conjunción><sustantivo>

Sin embargo, preferimos derivar reglas que expliquen la estructura como una colección de frases en tanto ello se pueda.

2.- <sujeto> → <frase nominal>

3.- <frase nominal> → <sustantivo>

4.- <frase nominal> → <sustantivo><conjunción><frase nominal>

Vemos que en esta regla <frase nominal> tiene a <frase nominal> como uno de sus elementos constitutivos, es decir tenemos una definición recursiva. Esta *propiedad recursiva* de las reglas gramaticales nos permite la representación finita de un conjunto infinito de sentencias.

Luego el sujeto del ejemplo se genera mediante la aplicación de las reglas 2 a 4 y una aplicación de cada una de estas reglas.

5.- <conjunción> → y

6.- <sustantivo> → Jorge

7.- <sustantivo> → Pablo

de la siguiente forma:

<sujeto>	
<frase nominal>	(regla 2)
<sustantivo><conjunción><frase nominal>	(regla 4)
<sustantivo> y <frase nominal>	(regla 5)
Jorge y <frase nominal>	(regla 6)
Jorge y <sustantivo>	(regla 3)
Jorge y Pablo	(regla 7)

Estas líneas, conocidas como *formas sentenciales*, conforman lo que se conoce como *derivación* de la frase sujeto de acuerdo a la aplicación de las reglas 2 a 7. Cada línea de la derivación resulta de la re-escritura de la línea precedente de acuerdo a alguna regla establecida.

El predicado de nuestro ejemplo consiste de un verbo y de una frase preposicional.

subieron a la terraza

verbo

frase preposicional

predicado

8.- <predicado> → <verbo><frase preposicional>

9.- <verbo> → subieron

La estructura de la frase preposicional es descripta mediante la siguiente regla:

10.- <frase preposicional> → <preposición><artículo><sustantivo>

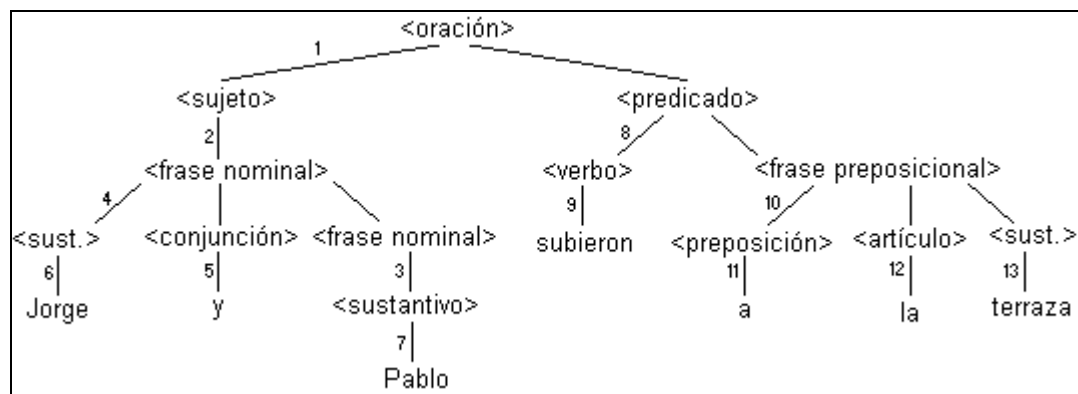
en la cual cada uno de los elementos constitutivos puede ser reemplazado por:

11.- <preposición> → a

12.- <artículo> → la

13.- <sustantivo> → terraza

Las reglas 1 a 13 constituyen una *gramática*. La oración que tomamos como ejemplo es un miembro del lenguaje generado por esta gramática, cuya *descripción estructural* puede ser representada por el siguiente diagrama en árbol.



Cada nodo de este árbol corresponde a la aplicación de una dada regla gramatical (ello se indica mediante los números). Dado que estas producciones especifican cómo un tipo de frase se forma mediante la combinación de otros tipos de frases, estas gramáticas se denominan *gramáticas descritas mediante estructuras de frases*.

Cada oración para la cual una dada gramática brinda una descripción estructural se conoce como una sentencia válida del lenguaje generado por la gramática. Por ejemplo, nuestra gramática también genera las siguientes oraciones:

Pablo y Jorge subieron a la terraza
Pablo y Pablo y Jorge subieron a la terraza
Pablo y Jorge y Pablo y Jorge subieron a la terraza

Estos ejemplos muestran cómo un conjunto finito de reglas, a través de sucesivas aplicaciones, puede generar un conjunto arbitrariamente grande de sentencias. Sin embargo, estas oraciones también son generadas por la gramática:

terrazza subieron a la Jorge
Pablo y terraza subieron a la Jorge

Las mismas son incorrectas semánticamente, pero verifican la gramática propuesta.

Un ejemplo del ALGOL

Como segundo ejemplo de una gramática descrita mediante estructuras de frases vamos a considerar un trozo de lenguaje ALGOL. Algol fue el primer lenguaje de programación práctico para el cual se formuló un conjunto completo de reglas gramaticales. Consideremos una forma simplificada de una sentencia de asignación del ALGOL, de la cual la frase:

$w := \text{if } x < y \text{ then } 0 \text{ else } x + y + 1$

es un ejemplo. Esta oración puede interpretarse de la siguiente forma: Si $x < y$ asigne valor 0 a w , de lo contrario asigne el valor $(x + y + 1)$ a w . Esta frase es escrita con símbolos que provienen del alfabeto:

$\{ w, x, y, 0, 1, <, :=, \text{if}, \text{then}, \text{else} \}$

que es solamente una parte de todo el alfabeto del ALGOL. Es importante notar que *if*, *then* y *else* se consideran símbolos simples, no divisibles.

$w := \text{if } x < y \text{ then } 0 \text{ else } x + y + 1$
parte izquierda expresión

1.- <declaración> $\rightarrow$ <parte izquierda><expresión>

El significado de <expresión> es el siguiente: <expresión> representa un valor numérico que debe asignarse a la variable definida en la parte izquierda. En este ejemplo, <expresión> es una construcción condicional formada de acuerdo a la regla gramatical número 3. Para la parte izquierda tenemos:



2.- $\langle \text{parte izquierda} \rangle \rightarrow \langle \text{identificador} \rangle :=$

3.- $\langle \text{expresión} \rangle \rightarrow \text{if } \langle \text{expresión booleana} \rangle \text{ then } \langle \text{expresión aritmética} \rangle \text{ else } \langle \text{expresión} \rangle$

Aquí la frase tipo expresión booleana es interpretada representando un valor de verdadero o falso. La construcción condicional se interpreta así: Si el valor de la expresión booleana es verdadero, su valor es el de la expresión aritmética, de lo contrario, su valor es el de $\langle \text{expresión} \rangle$. La recurrencia de la frase tipo expresión en esta regla permite que las frases tipo se aniden en una profundidad arbitraria.

La estructura de la frase:

$$x + y + 1$$

podría describirse de la siguiente forma:

$$\langle \text{expresión} \rangle \rightarrow \text{if } \langle \text{identificador} \rangle + \langle \text{identificador} \rangle + 1$$

sin embargo, pretendemos una representación más general, así tenemos:

4.- $\langle \text{expresión} \rangle \rightarrow \langle \text{expresión aritmética} \rangle$

De la misma forma podríamos indicar a:

$$x < y$$

$$\langle \text{expresión booleana} \rangle \rightarrow \langle \text{identificador} \rangle < \langle \text{identificador} \rangle$$

sin embargo, esta regla carece de generalidad y por lo tanto proponemos la siguiente en su reemplazo:

5.- $\langle \text{expresión booleana} \rangle \rightarrow \langle \text{expresión aritmética} \rangle < \langle \text{expresión aritmética} \rangle$

Pero una expresión booleana también puede tener la siguiente forma:

6.- $\langle \text{expresión booleana} \rangle \rightarrow \langle \text{expresión aritmética} \rangle = \langle \text{expresión aritmética} \rangle$

Siguiendo ahora con la regla 4, y considerando la expresión

$$x + y + 1$$

podemos seguir el desarrollo de la siguiente manera:

7.- $\langle \text{expresión aritmética} \rangle \rightarrow \langle \text{término} \rangle + \langle \text{expresión aritmética} \rangle$ (regla recursiva)

y

8.- $\langle \text{expresión aritmética} \rangle \rightarrow \langle \text{término} \rangle$

La frase tipo $\langle \text{término} \rangle$ se interpreta como un número o como letra que identifica una variable ($\langle \text{identificador} \rangle$); luego el valor de un término es el número en sí mismo o el valor de la variable. La regla 7, que es recursiva con respecto a la frase tipo $\langle \text{expresión aritmética} \rangle$, permite la generación de frases que contengan un número arbitrario de frases tipo $\langle \text{término} \rangle$ separadas por el símbolo +. La aparición de números y letras que indican variables requiere del desarrollo de las siguientes reglas:

9.- $\langle \text{término} \rangle \rightarrow \langle \text{identificador} \rangle$

10.- $\langle \text{término} \rangle \rightarrow 0$

11.- $\langle \text{término} \rangle \rightarrow 1$

En forma más general, un término puede ser también una expresión encerrada entre paréntesis:

12.- $\langle \text{término} \rangle \rightarrow (\langle \text{expresión} \rangle)$

Asimismo, como identificadores podemos incluir las tres letras que aparecen en nuestro ejemplo:

13.- $\langle \text{identificador} \rangle \rightarrow w$

14.- $\langle \text{identificador} \rangle \rightarrow x$

15.- $\langle \text{identificador} \rangle \rightarrow y$

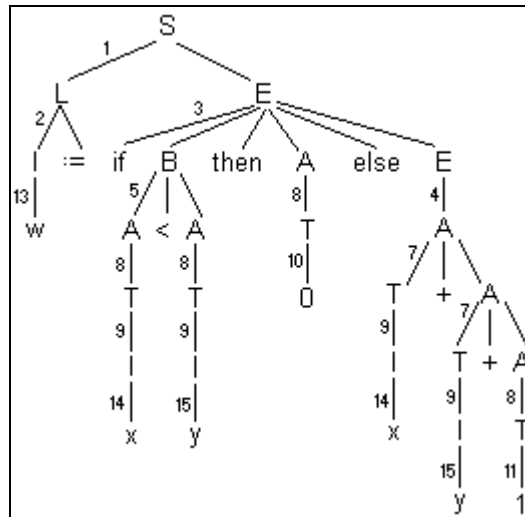
Para simplificar nuestro tratamiento vamos a introducir una convención notacional que se va a emplear a lo largo del estudio de las gramáticas formales: Se emplearán letras mayúsculas para indicar frases tipo:

S	$\langle \text{Statement} \rangle$	$\langle \text{declaración} \rangle$
L	$\langle \text{Left Part} \rangle$	$\langle \text{parte izquierda} \rangle$
E	$\langle \text{Expression} \rangle$	$\langle \text{expresión} \rangle$



I	<Identifier>	<identificador>
B	<Boolean>	<expresión booleana>
A	<Arithmetic>	<expresión aritmética>

Luego, el árbol que indica el análisis sintáctico de nuestra declaración de asignación es:



Expresada en función de estas letras, una gramática para frases del tipo <declaración> está definida por el siguiente conjunto de reglas:

$S \rightarrow LE$
 $L \rightarrow I \text{ :=}$
 $E \rightarrow \text{if } B \text{ then } A \text{ else } E$
 $E \rightarrow A$
 $B \rightarrow A < A$
 $B \rightarrow A = A$
 $A \rightarrow T + A$
 $A \rightarrow T$
 $T \rightarrow I$
 $T \rightarrow \emptyset$
 $T \rightarrow 1$
 $T \rightarrow (E)$
 $I \rightarrow w$
 $I \rightarrow x$
 $I \rightarrow y$

Este conjunto de reglas de re-escritura es una *gramática formal* en la cual las reglas se conocen como *producciones*. Con esta gramática se puede generar una gran variedad de declaraciones de asignación. La pregunta es ahora: ¿Sabemos que esta gramática es incompleta, pero es una representación adecuada de la declaración de asignación del ALGOL? La respuesta es: Cada string generado por la gramática es una frase legal de acuerdo a la sintaxis "oficial" del ALGOL.

Pero la gramática genera frases como

$x \text{ := } x$
 $x \text{ := if } w < w \text{ then } x + y \text{ else } x$

que no tienen ningún sentido en un programa computacional, pero que son aceptadas por la mayoría de los compiladores del ALGOL.

Veremos que es posible construir varias descripciones estructurales de una frase de acuerdo a un dado conjunto de producciones. Esa gramática es entonces una representación *ambigua* del lenguaje y puede conducir a una traducción incorrecta por parte de una persona. Problemas relacionados con la *ambigüedad* son por lo tanto de interés primordial en el diseño de un lenguaje.



Gramáticas Formales

Una gramática tiene tres componentes fundamentales:

- El *alfabeto* o colección de símbolos a partir del cual se construyen las sentencias que describen el lenguaje. Estas se denominan *letras o símbolos terminales de la gramática*, dado que la generación de una sentencia a través de la aplicación de reglas gramaticales debe terminar en un string que contiene sólo estos símbolos.
- Un conjunto de símbolos que indiquen o denoten frases llamados *letras o símbolos no terminales*.
- Un conjunto o colección de *reglas gramaticales o producciones*.
- Para completar la especificación del lenguaje, es necesario un punto inicial para la aplicación de las producciones. El símbolo S , conocido como *símbolo sentencia o símbolo distinguido*, se reserva para esta función en nuestro trabajo con gramáticas formales.

Vamos a comenzar ahora con una definición más general de gramática formal que excede los requerimientos presentados en ejemplos anteriores.

Definición 1: Una gramática formal es una cuaterna:

$$G = (N, T, P, S)$$

donde

- N Conjunto finito de símbolos no terminales
- T Conjunto finito de símbolos terminales
- $N \cap T = \emptyset$, N y T son disjuntos
- P Conjunto finito de producciones
- S Símbolo sentencia o elemento distinguido; $S \notin N \cup T$

Cada producción en P es un par ordenado de strings (α, β)

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

tales que φ, ω, ψ son strings en $(N \cup T)^*$ - inclusive vacíos - y $A \in (N \cup S)$; S o un símbolo no terminal.

Vamos a escribir una producción (α, β) como

$$\alpha \rightarrow \beta$$

Las gramáticas formales definidas aquí se conocen también como gramáticas descriptas mediante estructuras de frase. El proceso de generación de una oración de acuerdo a una gramática formal es el proceso de re-escritura sucesiva de formas sentenciales a través del uso de producciones de la gramática, comenzando con la forma sentencial S . La secuencia de formas sentenciales requeridas para generar una sentencia se conoce como *derivación* de la *sentencia* de acuerdo a la gramática.

Definición 2: Sea G una gramática formal. Un string de símbolos en $(N \cup T)^* \cup \{S\}$ se conoce como forma sentencial. Si $\alpha \rightarrow \beta$ es una producción de G y $\omega = \varphi \alpha \psi$ y $\omega' = \varphi \beta \psi$ son formas sentenciales, se dice que ω' se deriva inmediatamente de ω en G . Esta relación se indica de la siguiente forma:

$$\omega \Rightarrow \omega'$$

Si $\omega_1, \omega_2, \dots, \omega_n$ es una secuencia de formas sentenciales tales que $\omega_1 \Rightarrow \omega_2 \Rightarrow \dots \Rightarrow \omega_n$, se dice que ω_n es *derivable* de ω_1 . Esta relación se indica de la siguiente forma:

$$\omega_1 \overset{*}{\Rightarrow} \omega_n$$

La secuencia $\omega_1, \omega_2, \dots, \omega_n$ se denomina derivación de ω_n a partir de ω_1 de acuerdo a la gramática G .

Definición 3: El lenguaje $L(G)$ generado a partir de la gramática formal G es el conjunto de strings terminales derivables a partir de S

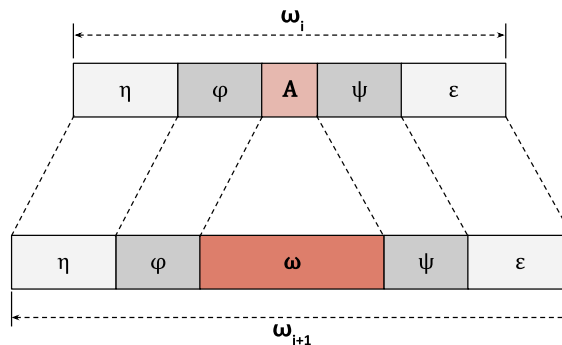


$$L(G) = \{\omega \in T^*/S \Rightarrow^* \omega\}$$

Si $\omega \in L(G)$, decimos que ω es un string, una sentencia o una palabra en el lenguaje generado por G .

De acuerdo a las definiciones, cada producción de una gramática formal es una regla gramatical que permite la modificación de una forma sentencial mediante la substitución de una letra particular que denota una frase por un string arbitrario.

La aplicación de la producción $\varphi A \psi \rightarrow \varphi \omega \psi$ para re-escribir la forma sentencial ω_i como ω_{i+1} se ilustra a continuación.



La aplicación de esta producción muestra dos generalizaciones que van más allá de los conceptos empleados en los ejemplos:

- 1: La substitución está condicionada a que A aparezca en un contexto particular definido por los strings φ y ψ de la producción.
- 2: El string ω que substituye a A puede ser el string vacío. Esto permite el borrado arbitrario de ciertos símbolos no terminales durante el curso de una producción. Esta segunda generalización permite que una forma sentencial decrezca en longitud en una etapa de derivación.



Sin embargo, parecería que es posible definir clases de gramáticas más generales. Por ejemplo, podríamos permitir producciones del tipo $\alpha \rightarrow \beta$ en las cuales $\alpha, \beta \in (N \cup T)^*$ sin ningún otro tipo de restricción (se conocen como Semi-Thue system)

Ejemplo: Una gramática G_1 que genera un lenguaje de tipo $\{a^k b^k / k \geq 1\}$ podemos escribirla como:

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow aSb \\ A &\rightarrow ab \end{aligned}$$

- Derivación de ab : $S \Rightarrow A \Rightarrow ab$
- Derivación de $a^2 b^2$: $S \Rightarrow A \Rightarrow aAb \Rightarrow aabb$
- Derivación de $a^3 b^3$: $S \Rightarrow A \Rightarrow aAb \Rightarrow aaAbb \Rightarrow aaabbb$

Luego, por inducción se verifica que $L(G) = \{a^k b^k / k \geq 1\}$



Tipos de Gramáticas

Mediante la restricción de la forma de las producciones permitidas en una dada gramática, es posible especificar cuatro tipos importantes de gramáticas formales, con sus correspondientes clases de lenguajes. La tabla siguiente muestra estas restricciones:

Tipo	Formato	Comentarios	
0	$\varphi A \psi \rightarrow \varphi \omega \psi$	No Restringida (Gramática que contrae)	
1	$S \rightarrow \lambda$ $\varphi A \psi \rightarrow \varphi \omega \psi$	Sensible al Contexto	
2	$S \rightarrow \lambda$ $A \rightarrow \omega$	Libres de Contexto	
3	$S \rightarrow \lambda$ $A \rightarrow aB$ $A \rightarrow a$	RLD	Regulares
	$S \rightarrow \lambda$ $A \rightarrow Ba$ $A \rightarrow a$	RLI	

En esta tabla se verifica que:

$$\begin{aligned}\varphi, \omega, \psi &\in (N \cup T)^* \\ A &\in (N \cup S) \\ B &\in N \\ a &\in T\end{aligned}$$

Moviéndonos en la tabla, de arriba hacia abajo, la forma de las producciones permitidas es cada vez más restringida. Es obvio que la gramática de un dado tipo pertenece también a los tipos de gramáticas ubicados más arriba en la tabla. Empleando la notación G_i para indicar una clase de gramáticas de tipo i , tendremos:

$$G_0 \supset G_1 \supset G_2 \supset G_3$$

En consecuencia, cada gramática debe poder generar lenguajes que sean subclases de los lenguajes generados por cualquier gramática que se encuentra en estratos superiores de la tabla. Si L_i denota la clase de lenguajes generados por las gramáticas incluidas en la clase G_i , luego:

$$L_0 \supseteq L_1 \supseteq L_2 \supseteq L_3$$

Con el conocimiento que disponemos hasta el presente no podemos concluir que estas inclusiones sean inclusiones propias. Esto lo verificaremos más adelante a medida que avancemos en el estudio de las gramáticas y los lenguajes.

Gramáticas no restringidas. Tipo 0

Al no tener restricciones, esta clase de gramáticas formales tiene una tremenda generalidad. El agregado de formas más generales de reglas de reemplazo no agrega nada a la fuerza descriptiva que poseen estas gramáticas. Conjuntos de strings que requieran de esta generalidad para su representación no presentan gran interés como lenguajes artificiales. Sin embargo estos sistemas de reglas de sustitución son de gran interés para los logistas en los estudios del proceso de deducción.



Debemos notar que la aplicación de la regla gramatical $\varphi A \psi \rightarrow \varphi \omega \psi$ en la etapa de derivación $\omega_i \Rightarrow \omega_{i+1}$ produce una nueva forma sentencial ω_{i+1} que tiene una menor longitud que ω_i . Estas reglas se conocen como producciones que producen contracción. Por ejemplo, una regla de este tipo:

$$bC \rightarrow b$$

Una gramática que tiene al menos una de estas producciones se conoce como gramática que produce contracción. Sólo las gramáticas de este tipo 0 pueden tener estas características.

Si G es una gramática no restringida (tipo 0), luego $L(G)$ es un lenguaje *recursivamente enumerable* (Tipo 0).

Gramáticas Sensibles al Contexto. Tipo 1

Una gramática sensible al contexto $G = (N, T, P, S)$ es una gramática formal en la cual todas las producciones son de la forma $\varphi A \psi \rightarrow \varphi \omega \psi$ con $\omega \neq \lambda$.

Esta gramática puede también contener la producción $S \rightarrow \lambda$. Si G es una gramática sensible al contexto (tipo 1), luego $L(G)$ es un lenguaje sensible al contexto (Tipo 1).

Una producción $\alpha \rightarrow \beta$, que satisface $|\alpha| \leq |\beta|$ se conoce como producción que no produce contracción. Se requiere que cada producción de esta gramática no produzca contracción.

En consecuencia, las formas sentenciales en cualquier derivación sensible al contexto

$$\omega_1 \Rightarrow \omega_2 \Rightarrow \dots \Rightarrow \omega_n$$

no decrecen en longitud

$$|\omega_1| \leq |\omega_2| \leq \dots \leq |\omega_n|$$

De aquí se desprende que, si G es una gramática sensible al contexto, $\lambda \in L(G)$ si y sólo si G contiene la producción $S \rightarrow \lambda$. Sin esta producción, no sería posible generar el string vacío y tendríamos por lo tanto una teoría de lenguajes formales poco elegante.

Gramáticas Libres de Contexto. Tipo 2

En una gramática libre de contexto, se requiere que los strings φ y ψ , que denotan contexto en la producción $\varphi A \psi \rightarrow \varphi \omega \psi$, sean vacíos. Así, la posibilidad de reemplazar una letra o símbolo no terminal en una forma sentencial, es independiente de los símbolos adyacentes, es decir, es independiente del contexto.

Definición: Una gramática libre de contexto $G = (N, T, P, S)$ es una gramática formal en la cual todas las producciones poseen la siguiente forma

$$A \rightarrow \omega$$

donde $A \in (N \cup S)$ y $\varphi, \omega, \psi \in (N \cup T)^* - \{\lambda\}$

La gramática puede también contener la producción $S \rightarrow \lambda$. Si G es una gramática libre de contexto (tipo 2), luego $L(G)$ es un lenguaje libre de contexto (Tipo 2).

Nuevamente, $\lambda \in L(G)$ si y sólo si G contiene la producción $S \rightarrow \lambda$. La gramática G_1 del ejemplo de la sección anterior es libre de contexto, al igual que las correspondientes a los ejemplos vistos en la introducción.

En una gramática libre de contexto, si $\omega \in T^*$ es un string derivable a partir de A , decimos que ω es denotado por A , o que ω es una frase de tipo A . Emplearemos la notación:

$$L(G, A) = \{ \omega \in T^* / A \overset{*}{\Rightarrow} \omega \}$$

para indicar el conjunto de frases de tipo A .

Las gramáticas libres de contexto son comúnmente empleadas para representar lenguajes artificiales, en especial lenguajes de programación.

Gramáticas Regulares. Tipo 3

Las gramáticas regulares son una clase altamente restringida de las gramáticas libres de contexto en la cual el lado derecho de la producción puede contener como máximo un sólo símbolo no terminal.



Definición: Una producción de la forma $A \rightarrow aB$ o $A \rightarrow a$ donde $A \in (N \cup S)$, $B \in N$ y $a \in T$, se denomina producción lineal por derecha.

Definición: Una producción de la forma $A \rightarrow Ba$ o $A \rightarrow a$ donde $A \in (N \cup S)$, $B \in N$ y $a \in T$, es una producción lineal por izquierda.

Una gramática formal es lineal por derecha si contiene solamente producciones lineales por derecha. De la misma forma, una gramática es lineal por izquierda si contiene sólo producciones lineales por izquierda. Cada forma de gramática lineal puede contener la producción $S \rightarrow \lambda$. Las gramáticas lineales por derecha e izquierda se conocen como gramáticas regulares.

Si G es una gramática regular (tipo 3), luego $L(G)$ es un lenguaje regular (Tipo 3).

Ejemplo 4: Sean G_1 y G_2 dos gramáticas lineales; G_1 es una gramática lineal por derecha y G_2 lo es por izquierda. Poseen las siguientes producciones:

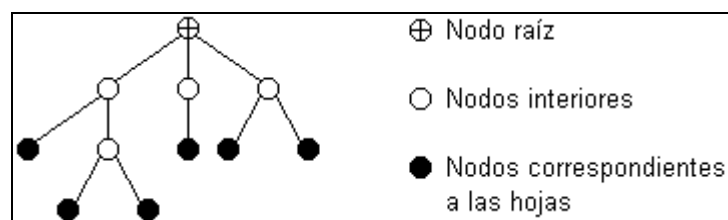
$G_1:$	$S \rightarrow 1B$	$G_2:$	$S \rightarrow B1$
	$S \rightarrow 1$		$S \rightarrow 1$
	$A \rightarrow 1B$		$A \rightarrow B1$
	$B \rightarrow 0A$		$B \rightarrow A0$
	$A \rightarrow 1$		$A \rightarrow 1$

Se puede verificar que: $L(G_1) = (10)^n 1 = 1(01)^n = L(G_2)$ con $n \geq 0$.

Árboles y Diagramas de Derivación

Los ejemplos ya desarrollados muestran cómo una derivación puede ser representada en forma de un diagrama en árbol que muestra claramente la estructura de una sentencia. Nos interesa ver ahora cómo estos diagramas en árbol se relacionan con las derivaciones de acuerdo a una gramática formal arbitraria.

Un *árbol* es interpretado como un grafo dirigido acíclico, en el cual cada nodo está conectado con el nodo que representa el símbolo distinguido (nodo raíz del árbol) por medio de un camino dirigido único.



Por convención se asume que los arcos están siempre dirigidos hacia abajo.

Un nodo dado de un árbol es llamado *descendiente* de aquellos nodos a partir de los cuales puede ser alcanzado por un camino dirigido. Así, el nodo raíz no es descendiente de ningún nodo, y los *nodos hojas* son aquellos que no poseen descendientes. Todos aquellos nodos que no son ni el nodo raíz ni nodos correspondientes a hojas se denominan *nodos interiores* del árbol.

El diagrama en árbol correspondiente a la derivación:

$$S \xRightarrow{*} \omega$$

con:

$$\omega \in (N \cup T)^*$$

en una gramática libre de contexto es un árbol en el cual el nodo raíz se identifica como S , los nodos interiores se asocian con símbolos pertenecientes a N y los nodos hojas se asocian con las letras de la forma sentencial ω . Para construir el diagrama en árbol se comienza con un árbol simple, que consiste sólo del nodo S , y que se va extendiendo en cada etapa de la derivación. En cada etapa de esta construcción, los nodos hojas del árbol (leídos de



izquierda a derecha) serán identificados con las letras de las formas sentenciales correspondientes. Consideremos una etapa de derivación arbitraria:

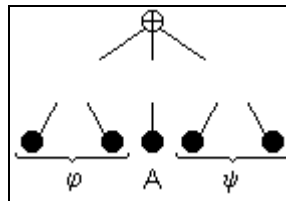
$$\varphi A \psi \Rightarrow \varphi B_1 B_2 \dots B_n \psi$$

en la cual se aplica la siguiente producción

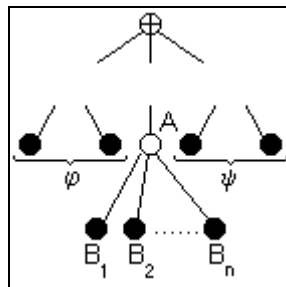
$$A \rightarrow B_1 B_2 \dots B_n$$

Donde $A \in (N \cup \{S\})$ y $B_i \in (N \cup T)^*$

Antes de aplicar esta producción, estamos en presencia de un árbol cuyos nodos hojas están indicados con las letras $\varphi A \psi$



Al aplicar la producción, el nodo A se transforma en un nodo interior que tiene como descendientes a los nodos $B_1 B_2 \dots B_n$. Luego, el diagrama en árbol resultante tiene los nodos hoja $\varphi B_1 B_2 \dots B_n \psi$.



Luego, el árbol de derivación se extiende mediante la aplicación de la producción:

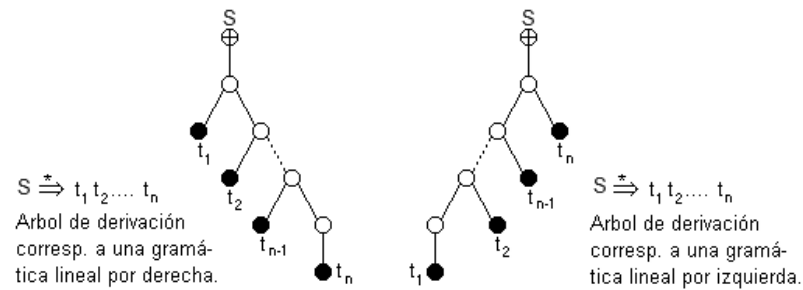
$$A \rightarrow B_1 B_2 \dots B_n$$

Ejemplo: Consideremos la gramática libre de contexto G

$$G \quad \left| \quad \begin{array}{l} S \rightarrow A \\ A \rightarrow AB \\ A \rightarrow B \\ B \rightarrow (A) \\ B \rightarrow () \end{array} \right.$$

El lenguaje generado por G se conoce como *lenguaje de paréntesis* L_p y consiste de todos los strings bien conformados que corresponden a juegos de paréntesis abiertos y cerrados que pueden surgir en expresiones matemáticas bien escritas. Por ejemplo, veamos la derivación correspondiente al string $()(())$.

Por cierto, una derivación que se efectúa de acuerdo a una gramática lineal por derecha siempre va a tener la forma ya vista en el ejemplo anterior. En los dos primeros casos la derivación se completa inmediatamente; en el tercero, la derivación debe continuar con la aplicación de producciones que tengan la siguiente forma: $A \rightarrow aB$ y finalizar con la aplicación de una producción de la forma $A \rightarrow a$. Los mismos conceptos se aplican a gramáticas lineales por izquierda. Los correspondientes árboles de derivación son:



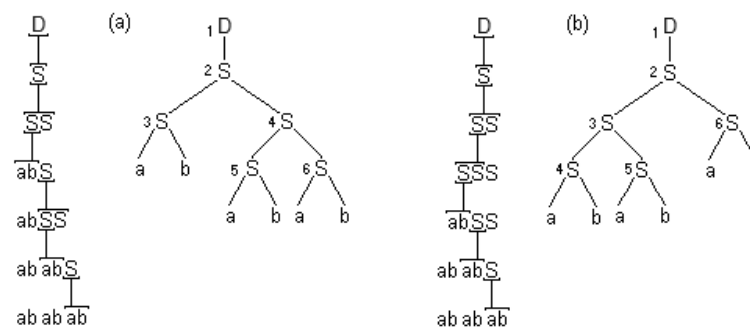
Ambigüedad

El siguiente ejemplo muestra que una gramática puede ser ambigua con respecto a alguna sentencia.

Ejemplo: Consideremos la siguiente gramática libre de contexto

$$G: \begin{array}{l} D \rightarrow S \\ S \rightarrow SS \\ S \rightarrow ab \end{array}$$

que puede producir las siguientes derivaciones del string *ababab*



Estas derivaciones se corresponden con dos árboles de derivación diferentes. La gramática G es ambigua con respecto a la sentencia *ababab*. Si los árboles de derivación se hubiesen empleado para darle significado al string generado, hubieran ocurrido interpretaciones erróneas.

Estamos interesados en determinar si una gramática genera sentencias ambiguas; luego, debemos definir precisamente lo que se entiende por derivación ambigua o por gramática ambigua. Si bien la noción de ambigüedad se aplica a todas las clases de gramáticas formales, nos vamos a limitar a la clase de gramáticas libres de contexto debido a que han probado ampliamente su utilidad en la representación de lenguajes de programación.

Definición: Sea G una gramática libre de contexto y sea ω una sentencia en el lenguaje generado por G . Luego, ω es ambiguo si existen derivaciones de ω que se corresponden con árboles de derivación diferentes.

A partir de la forma sentencial SS , existen dos caminos en los cuales el símbolo S ubicado más a la izquierda es reemplazado primero. El camino 1 corresponde al árbol (a), mientras que el 2 corresponde al árbol (b). Estos dos caminos tienen una propiedad única: “En cada etapa de la derivación se reescribe el símbolo no terminal ubicado más a la izquierda”. Estas derivaciones, conocidas como *derivaciones por izquierda*, permiten una definición precisa de las gramáticas ambiguas.

Definición: Sea G una gramática libre de contexto. Una derivación

$$\omega_0 \Rightarrow \omega_1 \Rightarrow \omega_2 \Rightarrow \dots \Rightarrow \omega_n$$

es una derivación por izquierda sí y sólo sí el símbolo no terminal ubicado más a la izquierda de ω_i es reemplazado para poder obtener ω_{i+1} . Esto se verifica para $0 \leq i < n$.



$$\omega_i = \varphi A \psi$$

$$\omega_{i+1} = \varphi \omega \psi$$

tal que $\varphi \in T^*$, $A \in N$ y $\psi \in (N \cup T)^*$ y $A \rightarrow \omega$ es una producción definida en G .

Los nodos interiores de los árboles del ejemplo anterior han sido numerados para indicar el orden en que las producciones se aplican en la derivación por izquierda. Dado que es fácil obtener un árbol de derivación a partir de cualquier derivación, la correspondencia entre árboles y derivaciones por izquierda es uno a uno. Cada oración generada por una gramática libre de contexto y asociada con un dado árbol, tiene una única derivación por izquierda.

Definición: Una gramática libre de contexto es ambigua sí y sólo si genera una dada sentencia a partir de dos o más derivaciones por izquierda distintas.

La gramática del último ejemplo es ambigua.

Hay dos maneras por las cuales puede surgir ambigüedad en una gramática libre de contexto:

1. Alguna oración tiene dos árboles de derivación estructuralmente diferentes.
2. Alguna sentencia tiene dos árboles de derivación estructuralmente similares, pero con diferentes rótulos en sus nodos interiores.

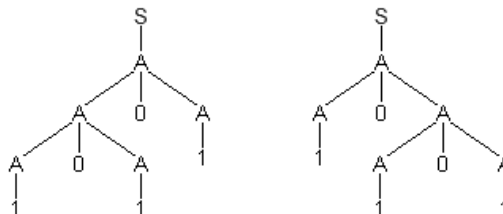
Estos dos casos se ilustran en los siguientes ejemplos.

Ejemplo: (Caso de Ambigüedad estructural)

Sea G :

$$\begin{array}{l} S \rightarrow A \\ A \rightarrow A0A \\ A \rightarrow 1 \end{array}$$

El string 10101 puede ser generado mediante dos caminos distintos.

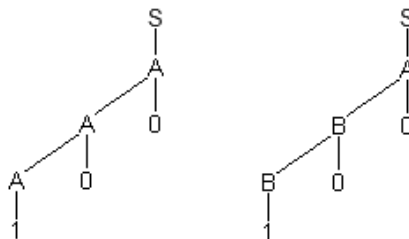


Ejemplo: (Caso de Ambigüedad asociada a rótulos)

Sea G :

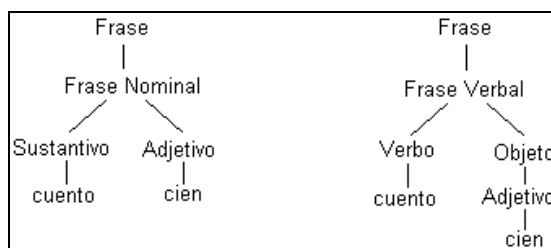
$$\begin{array}{l} S \rightarrow A \\ A \rightarrow B0 \\ A \rightarrow A0 \\ B \rightarrow B0 \\ A \rightarrow 1 \\ B \rightarrow 1 \end{array}$$

Las siguientes figuras muestran los árboles de derivación asociados con la derivación del string 100. Los árboles tienen idéntica estructura, pero los símbolos no terminales asignados a los nodos interiores son distintos.



Dado que en una gramática regular cada forma sentencial de una derivación puede contener solamente un único símbolo no terminal, la gramática será ambigua sólo si los árboles de derivación tienen distintos rótulos en los nodos interiores.

La ambigüedad es importante en el análisis de lenguajes, dado que el análisis de una sentencia con derivaciones ambiguas puede conducir a distintas asignaciones de significado. Un ejemplo de ambigüedad en el castellano es el siguiente:



Este tipo de ambigüedad es el que dificulta el proceso de traducción automática del lenguaje natural.